연습문제 정답과 풀이

학습 관련 기술 · 딥러닝개론및연습 2026년 2학기

강의노트 끝의 연습문제 17개의 정답과 풀이이다. 문제를 다시 적고 그 아래에 풀이를 쓴다. 코드를 실행하는 문제(5, 8, 11, 14, 17번)의 숫자는 모두 `연습문제_정답/code/exercise_runs.py`를 이 컴퓨터에서 실제로 실행한 결과이다. 초깃값과 미니배치 순서는 노트의 코드와 같은 씨앗(`np.random.seed(1)`, `np.random.seed(2)`)으로 고정하였다.

1. $f(x,y)=\frac{1}{20}x^2+y^2$에서 SGD가 발산하지 않는 학습률 α의 범위를 구하라. 또 이 범위 안에서 x 방향의 수렴이 가장 빠른 α와 그때 x가 한 단계에 몇 배가 되는지 구하라.

풀이. $\nabla f=\left(\frac{x}{10},\,2y\right)$이므로 SGD 갱신은 두 좌표가 따로따로이다.

$$x_{k+1}=x_k-\alpha\frac{x_k}{10}=\Big(1-\frac{\alpha}{10}\Big)x_k,\qquad y_{k+1}=y_k-2\alpha y_k=(1-2\alpha)\,y_k.$$

각 좌표는 매 단계 같은 수를 곱하는 등비수열이므로, 곱하는 수의 절댓값이 1 이하이면 발산하지 않는다.

$$\Big|1-\frac{\alpha}{10}\Big|\le 1 \iff 0\le\alpha\le 20,\qquad |1-2\alpha|\le 1 \iff 0\le\alpha\le 1.$$

둘 다 만족해야 하므로 $\boxed{0<\alpha\le 1}$이다. ($\alpha=1$이면 $y_{k+1}=-y_k$로 y가 $\pm y_0$ 사이를 진동하며 수렴하지 않고, $0<\alpha<1$이면 두 좌표 모두 0으로 수렴한다.) 경사가 급한 y 방향이 학습률의 상한을 정한다.
이 범위에서 x의 배율 $1-\frac{\alpha}{10}$은 α가 클수록 작으므로 $\alpha=1$에서 가장 빠르고, 그때 x는 한 단계에 $\boxed{0.9}$배가 된다. 즉 y 방향 때문에 학습률을 키울 수 없어서, 완만한 x 방향은 아무리 빨라도 한 단계에 10% 씩밖에 줄지 않는다. SGD가 이 함수에서 느린 이유이다.

2. 모멘텀에서 gradient가 매 단계 같은 값 $\boldsymbol{g}$로 일정하다고 하자. $\boldsymbol{v}_0=\mathbf{0}$일 때 $\boldsymbol{v}_k$를 구하고, $k\to\infty$일 때 $\boldsymbol{v}_k\to-\dfrac{\alpha}{1-\beta}\,\boldsymbol{g}$임을 보이라.

풀이. $\boldsymbol{v}_{k+1}=\beta\boldsymbol{v}_k-\alpha\boldsymbol{g}$를 차례로 쓰면

$$\boldsymbol{v}_1=-\alpha\boldsymbol{g},\quad \boldsymbol{v}_2=-\alpha\boldsymbol{g}(1+\beta),\quad \boldsymbol{v}_3=-\alpha\boldsymbol{g}(1+\beta+\beta^2),\quad \dots$$

이므로 (귀납법으로 확인된다)

$$\boldsymbol{v}_k=-\alpha\boldsymbol{g}\,(1+\beta+\cdots+\beta^{k-1})=-\alpha\,\frac{1-\beta^k}{1-\beta}\,\boldsymbol{g}.$$

$0\le\beta<1$이면 $\beta^k\to 0$이므로 $\boldsymbol{v}_k\to-\dfrac{\alpha}{1-\beta}\boldsymbol{g}$이다.
$\beta=0.9$이면 $\frac{1}{1-\beta}=10$이므로, gradient의 방향이 일정한 곳에서 모멘텀은 SGD보다 최대 10배 빠르게 움직인다. 반대로 gradient의 부호가 번갈아 바뀌는 방향에서는 $\boldsymbol{v}$에 쌓인 값이 서로 상쇄되어 진동이 줄어든다.

3. 시작점 $(-7,2)$에서 AdaGrad ($\alpha=1.5$)로 두 번째 단계 (x_2,y_2)를 손으로 계산하라 (ε은 무시한다). 표의 값 $(-4.573,\ 0.136)$과 비교하라.

풀이. 갱신식은 $\boldsymbol{h} \leftarrow \boldsymbol{h} + \boldsymbol{g} \odot \boldsymbol{g}$, $W \leftarrow W - \alpha\, \boldsymbol{g}/\sqrt{\boldsymbol{h}}$ (성분별)이고 $\nabla f = (x/10,\ 2y)$이다.
1단계. $\boldsymbol{g}_0 = (-0.7,\ 4)$, $\boldsymbol{h}_1 = (0.49,\ 16)$.

$$x_1 = -7 - 1.5 \cdot \frac{-0.7}{\sqrt{0.49}} = -7 + 1.5 = -5.5, \qquad y_1 = 2 - 1.5 \cdot \frac{4}{\sqrt{16}} = 2 - 1.5 = 0.5.$$

첫 단계에서는 $\sqrt{\boldsymbol{h}_1} = |\boldsymbol{g}_0|$이므로 두 좌표가 모두 정확히 $\alpha = 1.5$만큼 움직인다.
2단계. $\boldsymbol{g}_1 = (-0.55,\ 1)$, $\boldsymbol{h}_2 = (0.49 + 0.3025,\ 16 + 1) = (0.7925,\ 17)$.

$$x_2 = -5.5 + 1.5 \cdot \frac{0.55}{\sqrt{0.7925}} = -5.5 + \frac{0.825}{0.8902} \approx -5.5 + 0.9267 = -4.573,$$

$$y_2 = 0.5 - 1.5 \cdot \frac{1}{\sqrt{17}} = 0.5 - \frac{1.5}{4.1231} \approx 0.5 - 0.3638 = 0.136.$$

$\boxed{(x_2, y_2) \approx (-4.573,\ 0.136)}$으로 표의 값과 같다. y 방향은 첫 단계의 큰 gradient (4)가 h에 쌓여 있어서, $\boldsymbol{g}$가 1인데도 0.36만 움직인다. x 방향은 h가 작아서 gradient 0.55에 비해 크게 (0.93) 움직인다.

4. Adam에서 편향 보정을 하지 않으면 첫 단계의 이동량은 얼마가 되는가? ($\beta_1 = 0.9$, $\beta_2 = 0.999$, ε은 무시한다.) 편향 보정이 왜 필요한지 설명하라.

풀이. $\boldsymbol{m}_0 = \boldsymbol{v}_0 = \boldsymbol{0}$에서 첫 gradient g (성분 하나)를 받으면

$$m_1 = (1 - \beta_1) g = 0.1\, g, \qquad v_1 = (1 - \beta_2) g^2 = 0.001\, g^2.$$

보정하지 않으면 이동량은

$$\alpha \frac{m_1}{\sqrt{v_1}} = \alpha \frac{0.1\, g}{\sqrt{0.001}\, |g|} = \frac{0.1}{0.0316}\, \alpha \operatorname{sign}(g) \approx \boxed{3.16\, \alpha}$$

이다 (방향은 $-\operatorname{sign}(g)$). 보정하면 $\hat{m}_1 = m_1/(1-\beta_1) = g$, $\hat{v}_1 = v_1/(1-\beta_2) = g^2$이므로 이동량은 정확히 α이다.
편향 보정이 필요한 이유. $\boldsymbol{m}$, $\boldsymbol{v}$는 $\boldsymbol{0}$에서 시작하는 지수이동평균이므로 처음 몇 단계에서는 0 쪽으로 치우쳐 있다 (gradient가 일정하면 $m_k = (1-\beta_1^k)g$, $v_k = (1-\beta_2^k)g^2$). $\beta_2 = 0.999$가 $\beta_1 = 0.9$보다 1에 훨씬 가까워서 v가 훨씬 더 오래 작게 머문다. 그래서 분모 $\sqrt{v}$가 너무 작아지고, 학습 초기의 이동량이 의도한 α보다 커진다 (첫 단계에서 약 3배). $1 - \beta^k$로 나누면 $\hat{m}_k$, $\hat{v}_k$가 처음부터 g, g^2의 크기를 갖게 되어, 학습률 α가 처음부터 실제 이동량의 크기를 뜻하게 된다. k가 커지면 $1 - \beta^k \to 1$이므로 보정의 효과는 사라진다.

5. `mnist_optimizer_compare.py`에서 SGD의 학습률을 0.01에서 0.05, 0.1로 바꾸어 보라. SGD도 plateau를 빠져나오는가? 같은 조건에서 RMSProp ($\alpha = 0.001$)을 `optimizers`에 추가하여 네 기법과 비교하라.

풀이. 노트와 같은 조건 (은닉층 $[100] \times 4$, 초깃값 표준편차 0.05, 미니배치 128, 5 epoch, 같은 초깃값과 같은 미니배치)에서 실행한 검증 정확도이다.

기법	epoch 1	2	3	4	5
SGD ($\alpha = 0.01$)	0.1138	0.1144	0.2046	0.2824	0.4404
SGD ($\alpha = 0.05$)	0.4210	0.8404	0.9098	0.9274	0.9166
SGD ($\alpha = 0.1$)	0.7802	0.9134	0.9512	0.9584	0.9584
Momentum ($\alpha = 0.01, \beta = 0.9$)	0.8448	0.9286	0.9464	0.9588	0.9632
AdaGrad ($\alpha = 0.01$)	0.9448	0.9602	0.9660	0.9688	**0.9702**
RMSProp ($\alpha = 0.001, \rho = 0.9$)	0.9312	0.9498	0.9662	0.9690	0.9700
Adam ($\alpha = 0.001$)	0.9264	0.9548	0.9586	0.9668	0.9660

- **SGD도 plateau를 빠져나온다.** $\alpha = 0.01$에서는 2 epoch 동안 정확도가 11% (찍는 수준)에 머물렀지만, $\alpha = 0.05$에서는 2 epoch 만에 84%, $\alpha = 0.1$에서는 1 epoch 만에 78%가 되었다. plateau는 gradient가 아주 작은 평평한 영역이므로, 학습률을 키우면 같은 작은 gradient로도 더 멀리 움직여 빨리 빠져나온다. $\alpha = 0.1$의 SGD는 5 epoch 후 95.84%로 Momentum (96.32%)에 가깝다. 즉 노트의 비교에서 SGD가 크게 뒤진 것은 상당 부분 학습률 0.01이 SGD에게 너무 작았기 때문이다.
- $\alpha = 0.05$에서 epoch 4에서 5로 갈 때 정확도가 조금 떨어졌다 (0.9274 → 0.9166). 미니배치 gradient의 잡음 때문이며, epoch마다 값이 오르내릴 수 있다.
- **RMSProp**은 97.00%로 AdaGrad (97.02%)와 거의 같고 Adam (96.60%)보다 조금 높다. 첫 epoch부터 93%로, 적응적 학습률을 쓰는 세 기법은 모두 plateau 없이 바로 학습을 시작한다. 다만 5 epoch, 씨앗 하나의 결과이므로 0.1%p 수준의 차이로 순위를 매기기는 어렵다.

RMSProp은 `optimizers.py`에 이미 있는 `RMSProp(lr=0.001)`을 `optimizers` 사전에 한 줄 추가하면 된다.

6. 은닉층의 모든 가중치를 같은 값 c로 초기화하면 학습이 진행되어도 은닉층의 node들이 계속 같은 값을 가지는 이유를 역전파 식 $\frac{\partial L}{\partial W} = X^\mathsf{T}\frac{\partial L}{\partial A}$를 이용하여 설명하라.

풀이. 2층 network $A = XW^{(1)} + B^{(1)}$, $Z = h(A)$, $U = ZW^{(2)} + B^{(2)}$로 설명한다. $W^{(1)}$의 모든 성분이 c이고 $B^{(1)}$의 성분도 모두 같으면, $W^{(1)}$의 열이 모두 같다.
순전파. A의 j번째 열은 $XW^{(1)}_{:,j} + b_j$이므로 모든 열이 같다. 따라서 Z의 열 (은닉 node의 값)도 모두 같다.
역전파. $W^{(2)}$의 모든 성분도 c이면 $W^{(2)}$의 행이 모두 같으므로 $\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial U}W^{(2)\mathsf{T}}$의 열이 모두 같다. Z의 열이 같으므로 $h'(A)$의 열도 같고, $\frac{\partial L}{\partial A} = \frac{\partial L}{\partial Z} \odot h'(A)$의 열도 모두 같다. 그러면

$$\frac{\partial L}{\partial W^{(1)}} = X^\mathsf{T}\frac{\partial L}{\partial A}$$

의 j번째 열은 $X^\mathsf{T}\left(\frac{\partial L}{\partial A}\right)_{:,j}$이므로 **모든 열이 같다.** 마찬가지로 $\frac{\partial L}{\partial B^{(1)}}$의 성분도 모두 같다. 또 $\frac{\partial L}{\partial W^{(2)}} = Z^\mathsf{T}\frac{\partial L}{\partial U}$의 행은 Z의 열로 정해지므로 모든 행이 같다.
따라서 갱신 $W \leftarrow W - \alpha\frac{\partial L}{\partial W}$ 후에도 $W^{(1)}$의 열은 서로 같고 $W^{(2)}$의 행도 서로 같다. 같은 논리가 매 단계 반복되므로 은닉 node들은 학습 내내 같은 값을 가진다. 은닉층에 node가 100개 있어도 node 1개와 같은 표현력밖에 없다. 초깃값을 무작위로 정하는 것은 이 대칭을 깨기 위해서이다. ($c = 0$이면 $\frac{\partial L}{\partial Z} = 0$이 되어 $W^{(1)}$은 아예 갱신되지 않는다.)

7. $a = \sum_{i=1}^{n} z_i w_i$ 에서 $z_i = \max(0, a_i')$ 이고 a_i' 이 평균 0 인 대칭 분포를 따를 때 $\mathbb{E}[z_i^2] = \frac{1}{2}\mathrm{Var}(a_i')$ 임을 보이고, 이로부터 He 초깃값 $\mathrm{Var}(w) = \frac{2}{n}$ 을 유도하라.

풀이. a_i' 의 확률밀도를 p 라 하면 $p(-s) = p(s)$ 이다. $s < 0$ 이면 $z_i = 0$ 이므로

$$\mathbb{E}[z_i^2] = \int_0^\infty s^2 p(s)\, ds = \frac{1}{2}\int_{-\infty}^{\infty} s^2 p(s)\, ds = \frac{1}{2}\mathbb{E}[(a_i')^2] = \frac{1}{2}\mathrm{Var}(a_i').$$

(둘째 등호는 $s^2 p(s)$ 가 짝함수이기 때문이고, 마지막 등호는 평균이 0 이기 때문이다.)
w_i 는 z_i 와 독립이고 평균 0, 분산 $\mathrm{Var}(w)$ 이며, 항들이 서로 독립이라 하자. $\mathbb{E}[z_i w_i] = \mathbb{E}[z_i]\mathbb{E}[w_i] = 0$ 이므로

$$\mathrm{Var}(a) = \sum_{i=1}^{n} \mathrm{Var}(z_i w_i) = \sum_{i=1}^{n} \mathbb{E}[z_i^2]\,\mathbb{E}[w_i^2] = n \cdot \frac{1}{2}\mathrm{Var}(a') \cdot \mathrm{Var}(w).$$

(z_i 의 평균은 0 이 아니므로 $\mathrm{Var}(z_i)$ 가 아니라 $\mathbb{E}[z_i^2]$ 이 들어간다는 점에 주의한다.) 층을 지나도 활성화 전 값의 분산이 유지되려면 ($\mathrm{Var}(a) = \mathrm{Var}(a')$)

$$\frac{n}{2}\mathrm{Var}(w) = 1 \quad \Longrightarrow \quad \boxed{\mathrm{Var}(w) = \frac{2}{n}}, \qquad \text{즉 표준편차 } \sqrt{2/n}.$$

Xavier 초깃값 $\frac{1}{n}$ 과 비교하면, ReLU가 음수 절반을 0 으로 만들어 신호의 크기가 절반으로 줄어드는 것을 2 배로 보상한 것이다.

8. `weight_init_activation.py`에서 층의 수를 5 에서 20 으로 늘려 ReLU + Xavier와 ReLU + He의 마지막 층 활성화값의 표준편차를 비교하라. Xavier에서 표준편차가 층마다 약 몇 배씩 줄어드는가?

풀이. 노트의 코드와 같은 조건 (입력 1000×100, node 100 개, bias 0, `default_rng(0)`) 에서 층을 20 개로 늘린 결과이다. 처음 5 층의 값은 노트의 결과와 같다.

층	1	5	10	15	20	층당 배율 (기하평균)
ReLU + Xavier	0.583	0.177	0.0327	0.00812	**0.00198**	0.741
ReLU + He	0.825	1.000	1.047	1.469	**2.025**	1.048

마지막 (20 번째) 층의 표준편차는 Xavier가 0.00198, He가 2.03 으로 약 1000 배 차이가 난다. Xavier에서는 표준편차가 층마다 약 $\boxed{0.74\text{배}}$ 씩 줄어든다.
이론값은 $\frac{1}{\sqrt{2}} \approx 0.71$ 배이다. 문제 7 의 식에서 $\mathrm{Var}(w) = \frac{1}{n}$ 이면 $\mathrm{Var}(a) = \frac{n}{2} \cdot \frac{1}{n}\mathrm{Var}(a') = \frac{1}{2}\mathrm{Var}(a')$ 이므로 분산이 층마다 절반, 표준편차가 $\frac{1}{\sqrt{2}}$ 배가 된다. 20 층을 지나면 $(1/\sqrt{2})^{19} \approx 0.0014$ 배가 된다. 실험값이 조금 다른 것은 node가 100 개뿐이라 무작위 가중치에 따른 변동이 있기 때문이다. He는 이론적으로 층마다 1 배이며, 실험에서도 크게 줄거나 늘지 않는다 (층당 1.05 배로, 20 층 동안 0.8 에서 2.0 사이에 머문다). 층이 깊을수록 초깃값의 작은 차이가 거듭제곱으로 커지므로 ReLU에는 He 초깃값을 써야 한다.

9. 배치 정규화의 순전파 식 (미니배치 평균, 분산, 정규화, 확대와 이동)을 보고, 같은 미니배치에서 Affine 계층의 가중치 W 를 a 배 ($a > 0$) 하면 BatchNorm 계층의 출력 Y 가 변하지 않음을 보이라 (ε 은 무시한다). 이 사실로부터 $\frac{\partial L}{\partial (aW)} = \frac{1}{a}\frac{\partial L}{\partial W}$ 임을 설명하라.

풀이. BatchNorm의 입력을 $x_i = x_{\text{in},i}W + B$ (node 하나의 성분)라 하고, W를 aW로 바꾼 값을 x'_i라 하자. $x'_i = a\,x_{\text{in},i}W + B$이므로

$$\mu' = a\,\overline{x_{\text{in}}W} + B, \qquad x'_i - \mu' = a\big(x_{\text{in},i}W - \overline{x_{\text{in}}W}\big) = a\,(x_i - \mu).$$

(B는 모든 행에 같은 값이므로 평균을 빼면 사라진다.) 따라서

$$\sigma'^2 = \frac{1}{h}\sum_i (x'_i - \mu')^2 = a^2\sigma^2, \qquad \hat{x}'_i = \frac{x'_i - \mu'}{\sqrt{\sigma'^2}} = \frac{a(x_i - \mu)}{a\sigma} = \hat{x}_i,$$

이고 $y'_i = \gamma\hat{x}'_i + \beta = y_i$이다. 즉 W를 몇 배 하든 BatchNorm의 출력, 그리고 그 뒤의 손실 L도 그대로이다.

gradient. 위의 사실은 모든 W에 대하여 $L(aW) = L(W)$라는 뜻이다. 양변을 W로 미분하면 왼쪽은 연쇄법칙으로 $a \cdot \nabla L(aW)$이므로

$$a\,\nabla L(aW) = \nabla L(W) \qquad \Longrightarrow \qquad \frac{\partial L}{\partial (aW)} = \frac{1}{a}\frac{\partial L}{\partial W}.$$

(여기서 $\frac{\partial L}{\partial (aW)}$는 가중치가 aW인 점에서 구한 gradient이다.) 가중치가 커지면 ($a > 1$) gradient가 그만큼 작아지고, 작아지면 gradient가 커진다. 그래서 가중치의 크기가 저절로 조절되어, BatchNorm을 쓰면 초깃값의 크기나 학습률에 덜 민감해진다. 노트의 실험에서 초깃값의 표준편차를 바꾸어도 BatchNorm을 쓴 network가 잘 학습한 것과 같은 이야기이다.

10. 배치 정규화 계산 그래프의 역전파 단계 ②–⑦을 모두 대입하여 $\frac{\partial L}{\partial x_i} = \frac{1}{\sqrt{\sigma_B^2 + \varepsilon}}\big(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}}\big)$을 유도하라. ($g_i = \frac{\partial L}{\partial \hat{x}_i}$이고, $\bar{g}$, $\overline{g\hat{x}}$는 각각 g_k, $g_k\hat{x}_k$의 미니배치 평균이다.)

풀이. node 하나에 대하여 쓴다. $s = \sqrt{\sigma_B^2 + \varepsilon}$, $x_{c,i} = x_i - \mu$, $\hat{x}_i = x_{c,i}/s$이다.

② $g_i = \frac{\partial L}{\partial \hat{x}_i} = \gamma\frac{\partial L}{\partial y_i}$.

③ $\dfrac{\partial L}{\partial s} = -\sum_k g_k \dfrac{x_{c,k}}{s^2}$.

④ $\dfrac{\partial L}{\partial v} = \dfrac{1}{2s}\dfrac{\partial L}{\partial s} = -\dfrac{1}{2s^3}\sum_k g_k x_{c,k}$.

⑤ $\dfrac{\partial L}{\partial x_{c,i}} = \dfrac{g_i}{s} + \dfrac{2}{h}\,x_{c,i}\dfrac{\partial L}{\partial v} = \dfrac{g_i}{s} - \dfrac{x_{c,i}}{h\,s^3}\sum_k g_k x_{c,k}$.

$x_{c,i} = s\hat{x}_i$를 넣으면 $\dfrac{x_{c,i}}{hs^3}\sum_k g_k x_{c,k} = \dfrac{\hat{x}_i}{s}\cdot\dfrac{1}{h}\sum_k g_k\hat{x}_k = \dfrac{\hat{x}_i}{s}\,\overline{g\hat{x}}$이므로

$$\frac{\partial L}{\partial x_{c,i}} = \frac{1}{s}\big(g_i - \hat{x}_i\,\overline{g\hat{x}}\big).$$

⑥ $\dfrac{\partial L}{\partial \mu} = -\sum_k \dfrac{\partial L}{\partial x_{c,k}} = -\dfrac{1}{s}\Big(\sum_k g_k - \overline{g\hat{x}}\sum_k \hat{x}_k\Big) = -\dfrac{h}{s}\,\bar{g}$.

여기서 $\hat{x}$의 미니배치 평균이 0, 즉 $\sum_k \hat{x}_k = 0$임을 썼다.

⑦ $\dfrac{\partial L}{\partial x_i} = \dfrac{\partial L}{\partial x_{c,i}} + \dfrac{1}{h}\dfrac{\partial L}{\partial \mu} = \dfrac{1}{s}\big(g_i - \hat{x}_i\,\overline{g\hat{x}}\big) - \dfrac{1}{s}\bar{g}.$

따라서

$$\boxed{\frac{\partial L}{\partial x_i} = \frac{1}{\sqrt{\sigma_B^2 + \varepsilon}}\big(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}}\big).}$$

$\sum_i \hat{x}_i = 0$ 이고 $\sum_i \hat{x}_i^2 = h$ (ε 무시) 이므로 $\sum_i \frac{\partial L}{\partial x_i} = \frac{1}{s}(h\bar{g} - h\bar{g} - 0) = 0$ 이고 $\sum_i \hat{x}_i \frac{\partial L}{\partial x_i} = \frac{1}{s}(h\overline{g\hat{x}} - 0 - h\overline{g\hat{x}}) = 0$ 이다. 즉 입력 gradient에는 "모든 데이터에 같은 값을 더하는 방향"과 "$\hat{x}$ 방향으로 늘리는 방향" 성분이 없다. 정규화가 그 두 변화를 지워 버리기 때문이다.

11. `mnist_batchnorm.py`의 실험 2에서 `accuracy`가 이동평균 대신 미니배치 통계를 쓰도록 (`train_flg=True`) 바꾸고, 검증 데이터를 1장씩 넣어 정확도를 계산해 보라. 어떤 일이 생기는가? 왜 그런가?

풀이. 실험 2의 조건 (훈련 이미지 1000장, 은닉층 $[100]\times 5$, BatchNorm, SGD $\alpha = 0.01$, 20 epoch) 중 초깃값 표준편차 0.1인 network를 학습한 뒤 검증 데이터 5000장으로 정확도를 계산하였다.

추론에 쓴 통계	검증 정확도
이동평균 (`train_flg=False`, 원래 코드)	0.7030
미니배치 통계, 5000장을 한꺼번에	0.7038
미니배치 통계, 1장씩	**0.1000**

1장씩 넣으면 5000**장 모두를 같은 class (9)로 예측**하여 정확도가 10% (검증 데이터에서 9의 비율)로 떨어진다. 두 이미지의 점수 U를 비교하면 완전히 같다.
이유. 미니배치 크기가 $h = 1$이면 평균은 그 데이터 자신 ($\mu = x$)이므로

$$x - \mu = 0, \qquad \sigma^2 = 0, \qquad \hat{x} = \frac{0}{\sqrt{0+\varepsilon}} = 0, \qquad y = \gamma\cdot 0 + \beta = \beta.$$

첫 BatchNorm 계층의 출력이 입력과 관계없이 항상 β가 되므로, 그 뒤의 모든 계층의 값도 입력과 관계없이 같다. 즉 입력 이미지의 정보가 첫 BatchNorm에서 모두 지워진다. 5000장을 한꺼번에 넣으면 그 통계가 이동평균과 비슷하므로 정확도가 거의 같다. 하지만 그러면 한 이미지의 예측이 함께 넣은 다른 이미지에 따라 달라진다.
그래서 추론할 때는 학습 중에 모아 둔 이동평균 (`running_mean`, `running_var`)을 써야 한다. 그러면 데이터 한 장도 다른 데이터와 상관없이 예측할 수 있다. PyTorch에서는 `model.eval()`이 이 전환을 한다.

12. 가중치 감소를 적용한 SGD의 갱신식이 $W \leftarrow (1-\alpha\lambda)W - \alpha\frac{\partial L}{\partial W}$ 임을 보이라. $\alpha = 0.01$, $\lambda = 0.1$일 때 gradient가 0이라면 W가 처음 값의 절반이 되기까지 몇 번의 갱신이 필요한가?

풀이. 가중치 감소를 넣은 손실은 $\tilde{L} = L + \frac{\lambda}{2}\|W\|^2$ 이고 $\frac{\partial \tilde{L}}{\partial W} = \frac{\partial L}{\partial W} + \lambda W$ 이다. SGD로 갱신하면

$$W \leftarrow W - \alpha\Big(\frac{\partial L}{\partial W} + \lambda W\Big) = (1-\alpha\lambda)W - \alpha\frac{\partial L}{\partial W}.$$

gradient가 0이면 매번 W에 $1-\alpha\lambda = 1 - 0.001 = 0.999$를 곱한다. k번 갱신 후 $W_k =$

$0.999^k W_0$ 이므로 $0.999^k \le \frac{1}{2}$ 인 가장 작은 k 를 구한다.

$$k \ge \frac{\ln \frac{1}{2}}{\ln 0.999} = \frac{-0.693147}{-0.0010005} \approx 692.8 \quad \Longrightarrow \quad \boxed{693\text{번}}.$$

($\ln(1-x) \approx -x$ 를 쓰면 $k \approx \frac{\ln 2}{\alpha\lambda} = 693$ 으로 같은 답을 얻는다.) MNIST 에서 미니배치 100 이면 1 epoch 가 550 번의 갱신이므로, 약 1.3 epoch 동안 gradient 가 없는 가중치는 절반이 된다.

13. 하나의 parameter w 에 대하여 gradient 가 매 단계 g 로 일정하다고 하자 ($g \neq 0$, ε 은 무시). Adam 에 L2 규제를 쓰면 감소 항 λw 가 실제 갱신량에 주는 영향이 $|g|$ 에 따라 어떻게 달라지는지, AdamW 와 비교하여 설명하라.

풀이. Adam + L2. L2 규제는 손실에 $\frac{\lambda}{2}w^2$ 을 더하므로 Adam 이 받는 gradient 는 $g' = g + \lambda w$ 이다. gradient 가 일정하면 (편향 보정 후) $\hat{m} = g'$, $\hat{v} = g'^2$ 이므로 갱신량은

$$\Delta w = -\alpha \frac{\hat{m}}{\sqrt{\hat{v}}} = -\alpha \frac{g + \lambda w}{|g + \lambda w|} = -\alpha \,\mathrm{sign}(g + \lambda w)$$

이다. 갱신량의 크기는 항상 α 이고, 감소 항 λw 는 **분모 $\sqrt{\hat{v}}$ 로 함께 나누어진다.**

- $|g| \gg \lambda|w|$ 이면 $\mathrm{sign}(g+\lambda w) = \mathrm{sign}(g)$ 이므로 감소 항이 갱신량을 거의 바꾸지 못한다. 감소 항의 효과는 대략 $\alpha\lambda w/|g|$ 정도로, $|g|$ 에 반비례하여 작아진다.
- $|g| \ll \lambda|w|$ 이면 갱신이 $-\alpha\,\mathrm{sign}(w)$ 가 되어, w 가 매 단계 α 씩 0 으로 끌려간다.

즉 gradient 가 큰 parameter 일수록 규제를 덜 받는다. 규제의 세기가 parameter 마다 다르게, 의도와 상관없이 정해진다.
AdamW. 감소를 Adam 의 계산과 분리하여 따로 한다.

$$w \leftarrow w - \alpha\lambda w - \alpha \frac{\hat{m}}{\sqrt{\hat{v}}} = w - \alpha\lambda w - \alpha\,\mathrm{sign}(g).$$

감소량 $\alpha\lambda w$ 는 g 와 관계없이 모든 parameter 에서 같은 비율이다. SGD 에서 L2 규제와 가중치 감소가 같았던 성질 (문제 12) 을 Adam 에서도 되살린 것이다. 노트의 실험 (훈련 이미지 300 장) 에서 Adam + L2 ($\lambda = 0.1$) 는 시험 정확도가 72.90% 로 떨어졌지만 AdamW (0.1) 는 83.12% 였던 것도 이 차이에서 나온다.

14. `mnist_overfit.py` 에서 은닉층을 $[100] \times 6$ 대신 $[500, 500]$ 으로 바꾸고, 드롭아웃의 삭제 비율 $p = 0,\ 0.2,\ 0.5$ 를 **검증 데이터** 로 비교하라. 어떤 p 를 고르겠는가? 그 p 로 학습한 network 의 시험 정확도를 보고하라.

풀이. 노트의 조건 (훈련 이미지 300 장, He 초깃값, SGD $\alpha = 0.01$, 미니배치 100, 300 epoch, 같은 초깃값과 미니배치 순서) 에서 은닉층만 $[500, 500]$ 으로 바꾸었다.

p	훈련 정확도	검증 정확도	시험 정확도
0	1.0000	**0.8314**	(0.8309)
0.2	1.0000	0.8286	—
0.5	0.9800	0.8206	—

검증 정확도가 가장 높은 $\boxed{p = 0}$ (드롭아웃 없음)을 고른다. 그 network의 시험 정확도는 $\boxed{83.09\%}$이다. 시험 데이터는 p를 고른 **뒤에** 한 번만 계산하였고, 나머지 두 p의 시험 정확도는 계산하지 않았다 (계산해서 비교하면 시험 데이터로 고르는 것이 된다).
해석.

- 이 설정에서는 드롭아웃이 도움이 되지 않았다. 세 값의 차이는 1%p 이내 ($p = 0$과 0.2의 차이 0.28%p는 검증 이미지 14장)로 작다. 층이 2개뿐인 넓은 network는 깊은 $[100] \times 6$보다 덜 불안정하고, 학습률 0.01의 SGD로는 드롭아웃의 잡음이 학습을 늦추는 효과가 더 컸다고 볼 수 있다.
- 훈련 정확도는 모두 100% 근처이므로 오버피팅은 여전하다. 훈련 데이터가 300장뿐이라는 근본 문제는 규제만으로 해결되지 않는다.
- 검증 이미지 5000장에서 생기는 정확도의 표준오차는 약 $\sqrt{0.83 \cdot 0.17/5000} \approx 0.5\%$p이므로, 이 정도 차이로 "드롭아웃은 해롭다"고 결론 내릴 수는 없다. 씨앗을 바꾸어 여러 번 반복하면 더 믿을 만한 비교가 된다.

15. 하이퍼파라미터 10개를 시험하여 시험 정확도가 가장 높은 것을 골랐더니 시험 정확도가 98%였다. 이 98%를 처음 보는 데이터에 대한 성능이라고 보고해도 되는가? 왜 그런가? 어떻게 해야 하는가?

풀이. 보고하면 안 된다. 시험 데이터로 10개 중 하나를 **골랐으므로** 시험 데이터가 학습 과정 (모델 선택)에 쓰였다. 10개의 시험 정확도에는 각각 우연한 오차가 있는데, 그중 최댓값을 고르면 우연히 운이 좋았던 것을 고르게 된다. 그래서 98%는 처음 보는 데이터에 대한 성능보다 높게 (낙관적으로) 추정된 값이다. 하이퍼파라미터가 시험 데이터에 오버피팅된 것이라고 할 수 있다.
올바른 방법.

(1) 훈련 데이터의 일부를 **검증 데이터**로 떼어 놓는다 (노트에서는 60000장 중 5000장).

(2) 10개의 하이퍼파라미터를 훈련 데이터로 학습하고 **검증 정확도**로 비교하여 하나를 고른다.

(3) 고른 network의 시험 정확도를 **마지막에 한 번만** 계산하여 보고한다.

이미 시험 데이터로 골랐다면, 그 결과를 보고하는 대신 고른 하이퍼파라미터를 아직 쓰지 않은 새 데이터로 평가해야 한다.

16. 학습률 α를 $[10^{-5}, 1]$에서 (a) 그냥 균등하게, (b) $\alpha = 10^u$, $u \sim \text{Uniform}(-5, 0)$으로 뽑을 때, $\alpha < 10^{-2}$일 확률을 각각 구하라.

풀이. (a) $\alpha \sim \text{Uniform}(10^{-5}, 1)$이면

$$P(\alpha < 10^{-2}) = \frac{10^{-2} - 10^{-5}}{1 - 10^{-5}} = \frac{0.00999}{0.99999} \approx \boxed{0.0100}.$$

(b) $\alpha < 10^{-2} \iff u < -2$이므로

$$P(u < -2) = \frac{-2 - (-5)}{0 - (-5)} = \boxed{\frac{3}{5} = 0.6}.$$

(a)로 뽑으면 99%가 $[0.01, 1]$에 몰리고, $[10^{-5}, 10^{-4}]$, $[10^{-4}, 10^{-3}]$처럼 작은 구간은 거의 뽑히지 않는다. (b)는 $[10^{-5}, 10^{-4}], \ldots, [10^{-1}, 1]$의 다섯 구간(자릿수)을 각각 20%씩 고르게 뽑는다. 학습률의 효과는 값의 차이가 아니라 **비율**(몇 배인가)에 따라 달라지므로 로그 척도로 뽑는다.

17. `hyperparameter_search.py`의 결과를 보고 학습률과 가중치 감소의 범위를 좁혀(100번 대신 30번) 다시 탐색하라. 좁힌 범위와 그 근거, 가장 좋은 검증 정확도를 표로 정리하라.

풀이. 원래 탐색(100번)의 결과(노트의 결과를 다시 실행한 값).

- 학습률: 검증 정확도 상위 10개가 모두 $\alpha \in [0.11, 0.29]$에 있다. $\alpha < 10^{-2}$인 54번은 50 epoch 동안 학습이 느려서 최고 77.6%에 그쳤고, $\alpha \geq 0.3$인 12번은 평균 29%로 대부분 발산하였다. $[0.05, 0.3)$ 사이에서도 $\alpha > 0.25$인 4번 중 2번은 실패하였다(10.1%, 62.0%).
- 가중치 감소: 상위 10개의 λ가 10^{-6}에서 4×10^{-2}까지 고르게 퍼져 있다. $\alpha \in [0.05, 0.3)$인 20번만 보아도 λ와 검증 정확도 사이에 뚜렷한 관계가 없다.

좁힌 범위. 학습률은 $\boxed{\alpha \in [0.05, 0.25]}$(로그 척도)로 좁혔다. 가중치 감소는 이 실험에서 성능을 거의 좌우하지 않으므로 범위를 좁힐 근거가 없어서 $\lambda \in [10^{-6}, 10^{-1}]$(로그 척도)을 그대로 두었다. (근거 없이 좁히면 좋은 값을 놓칠 수 있다. 특정 λ 근처를 더 보고 싶다면 학습률을 고정하고 λ만 탐색하는 편이 낫다.)
결과(30번). 나머지 조건은 원래와 같다(훈련 이미지 500장, 은닉층 $[100] \times 6$, He, SGD, 50 epoch).

	원래(100번)	좁힌 범위(30번)
학습률 α의 범위	$[10^{-5}, 1]$	$[0.05, 0.25]$
가중치 감소 λ의 범위	$[10^{-6}, 10^{-1}]$	$[10^{-6}, 10^{-1}]$
가장 좋은 검증 정확도	0.8714	0.8710
검증 정확도의 평균	0.4602	0.8398
검증 정확도 50% 미만	54번	1번
고른 α, λ	$0.174, 5.3 \times 10^{-6}$	$0.0939, 2.8 \times 10^{-2}$
그 network의 시험 정확도	0.8701	0.8714

좁힌 범위의 상위 5개: $(\alpha, \lambda,$ 검증$) = (0.0939,\ 0.0283,\ 0.8710)$, $(0.109,\ 0.0280,\ 0.8668)$, $(0.103,\ 0.0096,\ 0.8660)$, $(0.151,\ 0.0018,\ 0.8646)$, $(0.125,\ 4.1 \times 10^{-5},\ 0.8646)$.
해석. 범위를 좁히자 30번 중 29번이 80% 이상이었고, 시도의 평균이 46%에서 84%로 올랐다. 즉 같은 계산량을 쓸모없는 영역에 낭비하지 않게 되었다. 하지만 가장 좋은 검증 정확도는 87.1%로 원래와 거의 같다. 이미 원래 탐색이 좋은 영역을 충분히 찾았고, 이 network와 훈련 이미지 500장으로 얻을 수 있는 정확도가 이 정도라는 뜻이다. 정확도를 더 올리려면 하이퍼파라미터보다 훈련 데이터의 양이나 network의 구조를 바꾸어야 한다. (시험 정확도는 고른 뒤 한 번만 계산하였다.)